Estructuras (registros)

- Colección de variables relacionadas, bajo un mismo nombre.
- Habitualmente usadas para definir registros que van a ser almacenados en archivos o en arreglos.

```
struct carta{
    int valor;
    char palo[4];
};
```

- Struct introduce la definición para la estructura *carta*
- Carta es el nombre de la estructura y se usa para declarar variables de ese tipo.
- Carta contiene dos campos: *valor* y *palo*.

- La definición de la estructura Carta *no reserva espacio en memoria*, simplemente crea un *nuevo tipo de dato* que puede ser usado para declarar variables de ese tipo de estructura.
- El *ámbito de visibilidad* de los campos de una struct se restringe a la propia definición del registro. Los campos de un registro pueden ser de cualquier tipo, ya sea *simple o compuesto*.

Declaración de variables tipo Struct

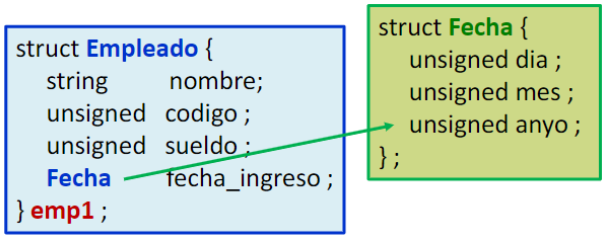
- *struct nombre_estructura{*
  / … /
  } nombre_variable1, nombrevariable2[tam];
- *nombre_estructura nombre_variable1, nombre_variable2[tam];*

Inicializaciones de variables tipo Struct

- **Mediante listas inicializadoras:**
  - Carta carta1 = {8, “Esp”};
- **Mediante sentencias de asignación:**
  - Carta carta2 = carta1;
- **Accediendo a sus campos mediante el operador punto(.):**
  - Carta carta1;
  - carta1.valor = 8;
  - strcpy(carta1.palo, “Esp”);

Entrada y salida de valores de tipo registro: *No existe un mecanismo predefinido para la lectura o escritura de structs*, por lo que el programador deberá ocuparse de la (lectura || escritura) de un registro efectuando la *(lectura || escritura) de cada uno de los campos de la struct por separado*.

Registros anidados o Estructuras Jerárquicas.



Operaciones válidas con structs.

- Acceder a los campos de una struct.
- Asignar una struct a otra struct del mismo tipo.
- Usar el operador *sizeof(struct);* para determinar el tamaño de una struct.
- Obtener la dirección (&) de una struct.

Acceso a campos de una struct.

- **Operador punto (.):**
  - cin >> carta1.palo; || cout << carta2.valor;
- **Operador flecha (->):** Se usa cuando se tienen punteros a variables struct.
  - Carta *ptrCarta1 = &Carta1;
  - cout << ptrCarta1 -> palo;

Uso de structs con funciones.

- **Paso de struct a funciones**
  - Paso de una struct completa (por defecto se pasan por valor)
  - Paso de campos de la struct